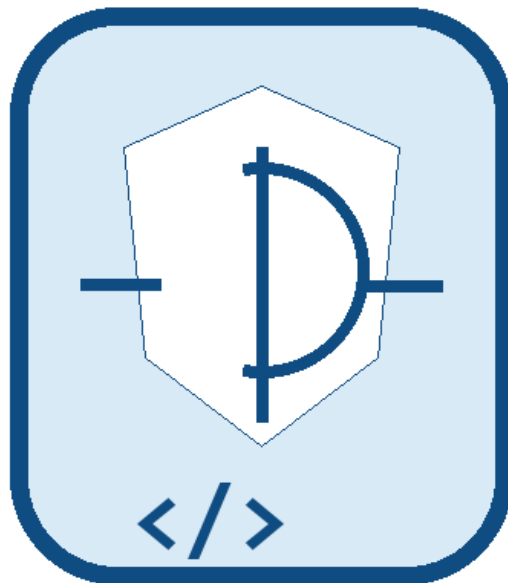




PLATAFORMA DE AUDITORÍA DE CÓDIGO BASADA EN INTELIGENCIA ARTIFICIAL

Trabajo Práctico Final
Programación de Vanguardia 2026

Laura Nicolino
Lucas Saputo
Nicolás Strasberg
Valentín Simaldone
Juan Ignacio Drovetto
Julio Severino

Ing. Alejandro Vázquez
Universidad de la Ciudad de Buenos Aires



Índice

Índice.....	3
Contexto	6
Problema a Resolver	6
Solución Propuesta.....	6
Beneficios	7
Innovación.....	7
Objetivo General.....	8
Objetivos Específicos.....	8
Funcionalidades Incluidas	9
Descripción General	10
• Frontend	10
• Backend Java.....	10
• Backend Python.....	10
• Base de Datos	10
Diagrama de Arquitectura	10
Descripción de Componentes	11
Frontend.....	11
Responsabilidades:	11
Backend Java	11
Responsabilidades:	11
Backend Python	11
Responsabilidades:	11
Base de Datos	11
Responsabilidades:	11
Frontend.....	12
Tecnologías Utilizadas	12
Estructura	12
Microservicio Java.....	12
Tecnologías Utilizadas	12
Responsabilidades	12
Microservicio Python.....	12
Tecnologías Utilizadas	12
Responsabilidades	12
Modelo Entidad Relación	13

Entidades	13
Usuario	13
Auditoria.....	14
Finding	14
Modelo Utilizado.....	15
Estrategía de Integración.....	15
Prompt Principal.....	15
Formato de Respuesta	15
Servicio Java	16
POST /api/auth/login	16
Descripción:	16
Request:.....	16
Response:	16
POST /api/audits	16
Descripción:	16
Request:.....	16
Response:	16
Servicio Python	16
POST /analyze	16
Descripción:	16
Request:.....	17
Response:	17
Autenticación	19
Autorización	19
Protección de Datos	19
Buenas Prácticas Aplicadas	19
Caso de Uso 1 – Auditoría de Código	20
Actor	20
Usuario	20
Flujo Principal.....	20
Resultado Esperado.....	20
Diagrama de secuencia:	20
Arquitectura de Contenedores	22
Docker Compose.....	22
Variables de Entorno.....	22
Pasos de Instalación	23

Pruebas Funcionales	24
Pruebas de Integración	24
Pruebas de IA	24
Comunicación Java ↔ Python	25
Integración con IA	25
Persistencia.....	25
Capturas de Pantalla	28

01

1. Introducción

Contexto del problema

Documentación técnica

**Contexto**

Actualmente los desarrolladores enfrentan desafíos relacionados con la calidad, seguridad y mantenibilidad del software.

La detección temprana de errores permite reducir costos de corrección y mejorar la calidad del producto final.

Problema a Resolver

El problema identificado es la falta de una herramienta integrada para revisar fragmentos de código con foco en seguridad, buenas prácticas y aprendizaje. En el flujo implementado, el usuario necesita autenticarse, enviar código en lenguajes soportados y recibir hallazgos trazables, explicaciones pedagógicas, código sugerido e historial persistente por usuario.

Solución Propuesta

La plataforma implementada combina un frontend React con editor Monaco, un backend Java Spring Boot protegido con JWT, un microservicio Python FastAPI para análisis IA/mock y una base SQL Server. El backend crea auditorías asincrónicas, estima costo por tokens, aplica rate limit, persiste resultados y expone historial, detalle y métricas por usuario.

02

2. Justificación del Proyecto

Valor e innovación

Documentación técnica



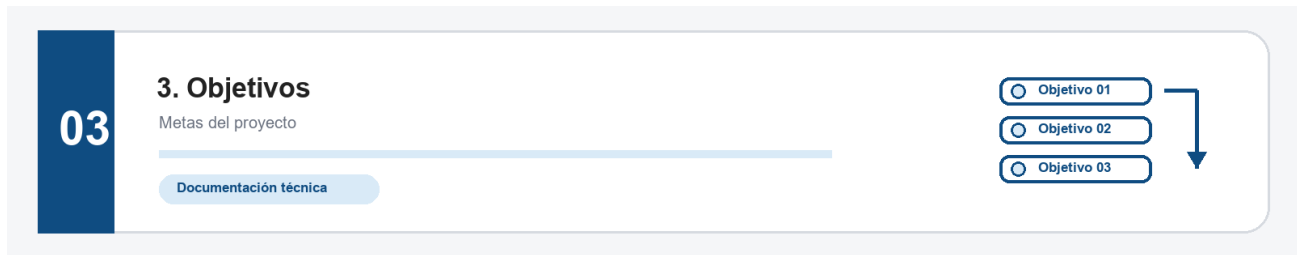
La propuesta consiste en desarrollar una plataforma capaz de analizar código fuente utilizando técnicas de Inteligencia Artificial y reglas de validación automatizadas.

Beneficios

- Detección temprana de vulnerabilidades.
- Mejora de la calidad del código.
- Asistencia pedagógica para desarrolladores.
- Escalabilidad mediante arquitectura de microservicios.

Innovación

La innovación principal es separar la experiencia de auditoria, la persistencia y la integración IA en servicios independientes. El sistema soporta modo mock para desarrollo reproducible y modo IA con OpenAI, valida el contrato mediante Pydantic, conserva la trazabilidad de cada auditoria por UUID y ofrece una devolución pedagógica orientada a estudiantes.



Objetivo General

Desarrollar una plataforma de auditoría inteligente de código fuente utilizando una arquitectura basada en microservicios.

Objetivos Específicos

- Detectar vulnerabilidades de seguridad.
- Identificar malas prácticas.
- Proponer refactorizaciones.
- Explicar errores de forma pedagógica.
- Soportar múltiples lenguajes de programación.

04

4. Alcance de la Solución

Funcionalidades y límites

Documentación técnica



Funcionalidades Incluidas

- Registro de usuarios.
- Inicio de sesión.
- Auditoría de código.
- Historial de análisis.
- Generación de recomendaciones.
- Explicaciones generadas mediante IA.

05

5. Arquitectura General

Microservicios y componentes

[Documentación técnica](#)

Descripción General

La solución fue diseñada bajo una arquitectura de microservicios compuesta por:

- **Frontend**
- **Backend Java**
- **Backend Python**
- **Base de Datos**
 - Servicio de Inteligencia Artificial

Diagrama de Arquitectura

Diagrama de arquitectura:

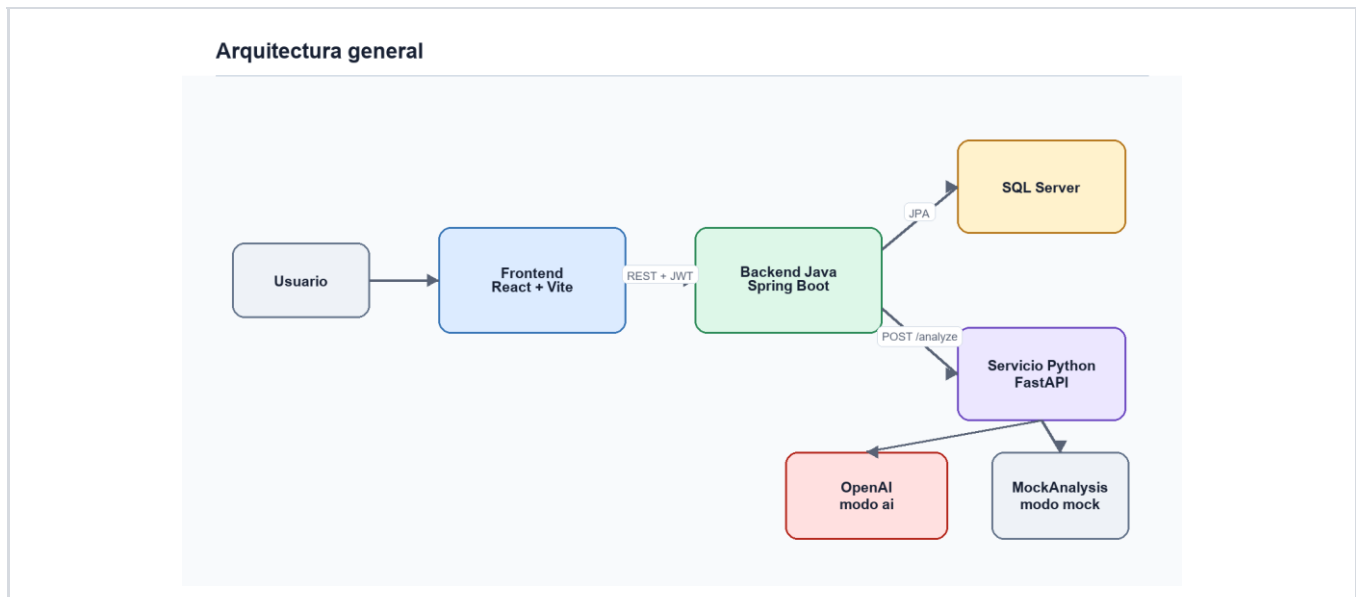


Figura 1 – Arquitectura general de la plataforma.

Descripción de Componentes

Frontend

Responsabilidades:

Responsable de la interfaz de usuario: registro, login, almacenamiento local del JWT, carga de código en Monaco Editor, selección de lenguaje, ejecución de auditorías, polling del detalle, visualización de hallazgos, historial, filtros y métricas.

Backend Java

Responsabilidades:

Responsable de exponer la API principal, autenticar con JWT, validar DTOs, normalizar lenguajes, aplicar rate limit y presupuesto estimado, crear auditorías asincrónicas, invocar el servicio Python, persistir usuarios/auditorías/hallazgos y devolver historial, detalle y métricas.

Backend Python

Responsabilidades:

Responsable de encapsular el análisis de código. Expone FastAPI, valida auditId/language/code con Pydantic, selecciona estrategia mock o IA según variables de entorno, construye prompts y devuelve un JSON compatible con el contrato que consume Java.

Base de Datos

Responsabilidades:

Responsable de persistir usuarios, auditorías y hallazgos. La implementación usa entidades JPA con tablas users, audits y findings, índices para consultas por usuario/fecha/lenguaje/riesgo y relaciones User 1-N Audit, Audit 1-N Finding.

06

6. Diseño de Componentes

Frontend, backend y servicio IA

Documentación técnica

**Frontend****Tecnologías Utilizadas**

React 18, Vite 5, Monaco Editor, lucide-react, Fetch API, Playwright para pruebas e2e y Docker/Nginx para distribución del frontend.

Estructura

La estructura principal está en Frontend/src: api.js centraliza las llamadas REST y main.jsx contiene la aplicación con vistas de autenticación, análisis, resultados, historial y métricas. Los assets visuales y estilos se concentran en styles.css.

Microservicio Java**Tecnologías Utilizadas**

Java 17, Spring Boot 3.2.5, Spring Web, Spring Data JPA, Spring Security, Bean Validation, RestTemplate, JWT, H2 para entorno local/test, SQL Server JDBC para Docker y Maven.

Responsabilidades

El microservicio Java funciona como API Gateway funcional de la plataforma: registra y autentica usuarios, protege endpoints, recibe auditorías, persiste el estado pending/processing/success/failed, consume Python con reintentos/circuit breaker y expone historiales y métricas.

Microservicio Python**Tecnologías Utilizadas**

FastAPI 0.115.6, Uvicorn, Pydantic, OpenAI SDK 1.59.6, python-dotenv, pytest y httpx.

Responsabilidades

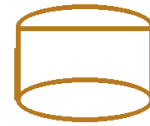
El microservicio Python valida solicitudes de análisis, ejecuta MockAnalysisService por defecto o AIAnalysisService en modo ai, construye prompts con JSON Schema, maneja errores controlados y devuelve findings normalizados con tipo, severidad, título, descripción, línea y sugerencia.

07

7. Diseño de Base de Datos

Modelo persistente

Documentación técnica



Modelo Entidad Relación

DER:

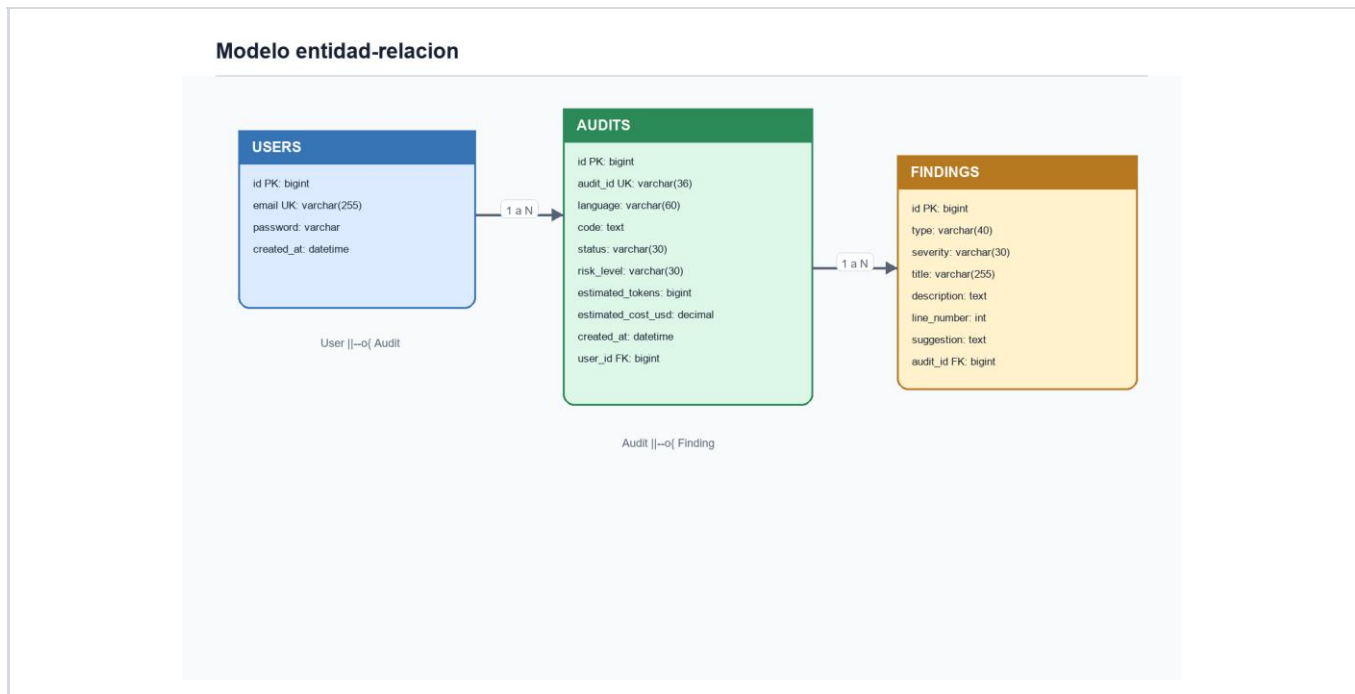


Figura 2 – Modelo entidad-relación.

Entidades

Usuario

Campo	Tipo	Descripcion
id	Long / bigint	Clave primaria autoincremental.
email	String / varchar(255)	Email unico usado como username e indice idx_users_email.
password	String	Hash BCrypt de la contrasena.
createdAt	LocalDateTime / datetime	Fecha de creacion generada en @PrePersist.

Auditoria

Campo	Tipo	Descripcion
id	Long / bigint	Clave primaria autoincremental.
auditId	String / varchar(36)	UUID publico unico de la auditoria.
userId	Long / FK	Usuario propietario de la auditoria.
language	String / varchar(60)	Lenguaje normalizado: java, python, javascript o json.
code	TEXT	Codigo fuente enviado.
status	String / varchar(30)	pending, processing, success o failed.
riskLevel	String / varchar(30)	unknown, low, medium o critical.
estimatedTokens	Long	Estimacion de tokens calculada por longitud/4.
estimatedCostUsd	BigDecimal	Costo estimado en USD.
createdAt	LocalDateTime	Fecha de creacion.

Finding

Campo	Tipo	Descripcion
id	Long / bigint	Clave primaria autoincremental.
auditId	Long / FK	Auditoria a la que pertenece el hallazgo.
type	String / varchar(40)	security, syntax, refactor o best_practice.
severity	String / varchar(30)	critical, warning o suggestion.
title	String / varchar(255)	Titulo corto del hallazgo.
description	TEXT	Descripcion del problema.
line	Integer	Linea afectada, nullable.
suggestion	TEXT	Recomendacion concreta.

08

8. Integración con Inteligencia Artificial

Análisis asistido por IA

Documentación técnica



Modelo Utilizado

El servicio Python esta preparado para usar OpenAI en modo ai con el modelo configurable AI_MODEL, cuyo valor por defecto en Settings es gpt-4o-mini. En desarrollo y pruebas usa modo mock por defecto para obtener respuestas determinísticas sin depender de credenciales externas.

Estrategia de Integración

Java invoca al servicio Python mediante RestTemplate contra PYTHON_AI_SERVICE_URL + /analyze. La auditoria se crea primero como pending, luego un worker asincronico la marca processing, llama a Python, calcula el riesgo general desde las severidades y persiste hallazgos; ante fallas marca la auditoria como failed con una explicacion controlada.

Prompt Principal

Prompt principal real usado por PromptBuilder:

Sistema: Act as a Senior Developer specialized in secure code reviews, clean code, refactoring, and pedagogy for programming students. Analyze the submitted code for security vulnerabilities, obvious syntax errors, bad practices, and refactoring opportunities. Return only valid JSON compatible with the AnalyzeResponse JSON Schema.

Usuario: auditId, language y code en bloque text. La respuesta debe conservar auditId, usar status='success' si finaliza, incluir findings o arreglo vacio, explicacion pedagogica y refactoredCode cercano al codigo original.

Formato de Respuesta

```
{
  "auditId": "uuid-generado-por-java",
  "status": "success | failed",
  "findings": [
    {
      "type": "security | syntax | refactor | best_practice",
      "severity": "critical | warning | suggestion",
      "title": "Titulo del hallazgo",
      "description": "Descripcion concreta",
      "line": 1,
      "suggestion": "Recomendacion"
    }
  ],
  "pedagogicalExplanation": "Explicacion para el usuario",
  "refactoredCode": "Codigo sugerido"
}
```

09

9. Especificación de APIs

Contratos REST

[Documentación técnica](#)

GET

POST

JWT

Servicio Java

POST /api/auth/login

Descripción:

Autentica un usuario existente. Valida email y password, verifica BCrypt contra la clave almacenada y devuelve un JWT firmado con fecha de expiracion.

Request:

Ver tabla de endpoints y contratos detallados debajo.

Response:

Ver tabla de endpoints y contratos detallados debajo.

POST /api/audits

Descripción:

Crea una auditoria autenticada. Valida lenguaje/codigo, aplica rate limit, estima tokens/costo, persiste una auditoria pending y dispara el analisis asincronico.

Request:

Ver tabla de endpoints y contratos detallados debajo.

Response:

Ver tabla de endpoints y contratos detallados debajo.

Servicio Python

POST /analyze

Descripción:

Endpoint interno del servicio Python que recibe auditId, language y code, ejecuta la estrategia mock o IA y responde con hallazgos normalizados, explicacion pedagogica y codigo refactorizado.

Request:

Ver tabla de endpoints y contratos detallados debajo.

Response:

Ver tabla de endpoints y contratos detallados debajo.

Endpoints detectados en el código fuente:

Método	Endpoint	Auth	Descripción
GET	/health	Publica	[Java] Devuelve status=ok y service=auditoria-backend.
POST	/api/auth/register	Publica	[Java] Request email/password. Crea usuario, valida email y password >= 8, responde 201 con id, email, createdAt.
POST	/api/auth/login	Publica	[Java] Request email/password. Responde token JWT, email y expiresAt; 401 ante credenciales invalidas.
POST	/api/audits	Bearer JWT	[Java] Request language/code. Crea auditoria pending, estima tokens/costo y dispara analisis asincronico. Responde 202 con auditId, status, riskLevel, findings vacio y estimacion.
GET	/api/audits	Bearer JWT	[Java] Query opcionales language, riskLevel, page, size. Devuelve resumen ordenado por createdAt desc.
GET	/api/audits/metrics	Bearer JWT	[Java] Devuelve totalAudits, totalFindings, byRiskLevel y byLanguage del usuario autenticado.
GET	/api/audits/{auditId}	Bearer JWT	[Java] Devuelve detalle de una auditoria del usuario: codigo, status, riesgo, hallazgos, explicacion, refactor y costos.
GET	/health	Interna/publica	[Python] Devuelve status=ok y service=code-audit-ai-service.
POST	/analyze	Interna desde Java	[Python] Request auditId/language/code. Devuelve auditId, status success/failed, findings,

			pedagogicalExplanation y refactoredCode.
--	--	--	---

10

10. Seguridad

JWT, validación y protección

[Documentación técnica](#)**Autenticación**

La autenticación se implementa con JWT. El login devuelve token, email y expiresAt; el frontend guarda el token en localStorage y lo envía como Authorization: Bearer <token>. JwtAuthenticationFilter valida firma, subject y expiración antes de poblar el SecurityContext.

Autorización

Los endpoints POST /api/auth/register, POST /api/auth/login y GET /health son públicos. El resto requiere usuario autenticado y las consultas de auditorías filtran siempre por el User autenticado, evitando acceder a auditorías de otros usuarios.

Protección de Datos

Las contraseñas se almacenan con PasswordEncoder/BCrypt, no en texto plano. El JWT se firma con jwt.secret configurable. La base conserva auditorías asociadas al usuario, y la configuración Docker obtiene credenciales y secretos desde .env.

Buenas Prácticas Aplicadas

Validación de DTOs con Bean Validation y Pydantic, CORS configurable, CSRF deshabilitado para API stateless, manejo centralizado de errores, rate limiting por usuario, presupuesto máximo estimado de tokens, reintentos y circuit breaker al invocar Python.

11

11. Casos de Uso

Flujo del usuario

Documentación técnica



Caso de Uso 1 – Auditoría de Código

Actor

Usuario

Flujo Principal

1. El usuario ingresa código fuente.
2. El sistema envía el código al backend.
3. El backend solicita análisis al servicio de IA.
4. El resultado es almacenado.
5. El usuario visualiza el informe.

Resultado Esperado

Diagrama de secuencia:

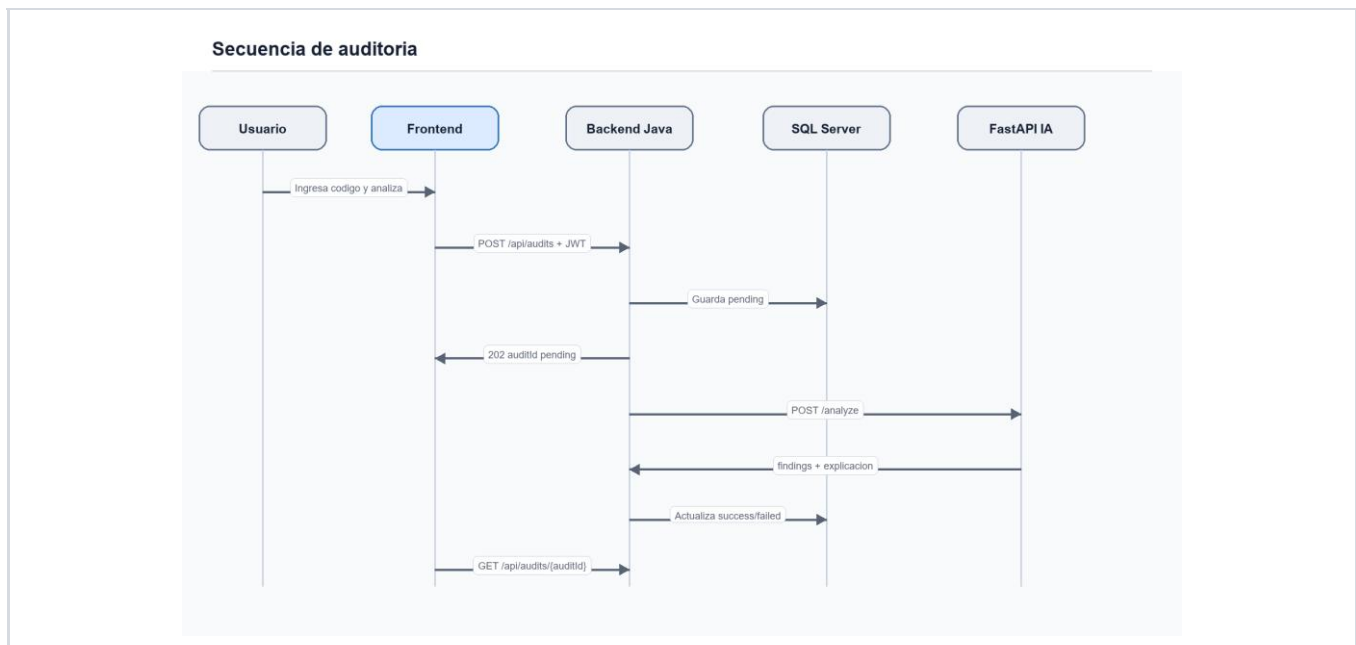


Figura 3 – Secuencia de auditoría.

El usuario obtiene una auditoria persistida con estado final success o failed. En caso exitoso visualiza riesgo general, cantidad de hallazgos, lineas afectadas, severidad, explicación pedagógica, recomendación y código refactorizado; tambien puede reabirla desde el historial.

12

12. Dockerización y Despliegue

Contenedores y entorno

Documentación técnica



Arquitectura de Contenedores

Diagrama de contenedores:

Arquitectura de contenedores

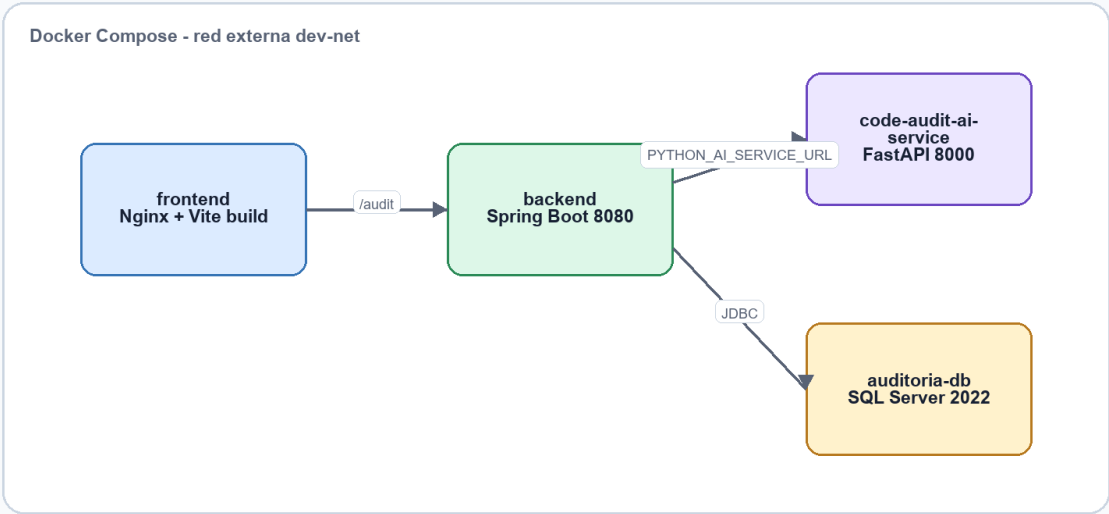


Figura 4 – Arquitectura de contenedores.

Docker Compose

La composición real se define en docker-compose.yml extendiendo infra/docker-compose.yml. Servicios: auditoria-db con SQL Server 2022 y volumen sqlserver-data; code-audit-ai-service construido desde code-audit-ai-service; backend construido desde backend con SPRING_PROFILES_ACTIVE=docker, PYTHON_AI_SERVICE_URL=http://code-audit-ai-service:8000 y datasource SQL Server; frontend construido desde Frontend con VITE_API_BASE_URL=/audit y VITE_BASE_PATH=/audit/.

Variables de Entorno

Variable	Descripción
SPRING_DATASOURCE_URL	Cadena JDBC real usada por el backend Docker para conectarse a SQL Server. Valor: jdbc:sqlserver://auditoria-db:1433;databaseName=master;encrypt=true;trustServerCertificate=true

SPRING_DATASOURCE_USERNAME	Usuario configurado para la conexión JDBC. Valor: sa
MSSQL_SA_PASSWORD	Password de SQL Server definida en el archivo .env. Valor: <definido en .env>
JWT_SECRET	Secreto HMAC utilizado para firmar tokens JWT. Valor: <secreto HMAC de al menos 32 caracteres>
JWT_EXPIRATION	Duración del token en milisegundos. Valor: 86400000
PYTHON_AI_SERVICE_URL	URL interna del servicio FastAPI consumida por Java. Valor: http://code-audit-ai-service:8000
ANALYSIS_MODE	Modo de análisis disponible: mock o ai. Valor: mock ai
AI_PROVIDER	Proveedor de IA configurado. Valor: openai
AI_MODEL	Modelo OpenAI configurado por defecto. Valor: gpt-4o-mini
AI_API_KEY	Clave opcional cuando ANALYSIS_MODE=ai. Valor: <opcional si ANALYSIS_MODE=ai>
VITE_API_BASE_URL	Base URL usada por el frontend para llamar al backend. Valor: /audit
VITE_BASE_PATH	Base path del frontend en despliegue. Valor: /audit/

Pasos de Instalación

1. Clonar repositorio.
2. Configurar variables.
3. Ejecutar Docker Compose.
4. Acceder a la aplicación.

13

13. Pruebas Realizadas

Validación funcional

[Documentación técnica](#)**Pruebas Funcionales**

Caso	Resultado
Login	Cubierto por MockMvc y Playwright: devuelve JWT, email y expiresAt; rechaza credenciales invalidas.
Auditoria	Cubierta por MockMvc: POST /api/audits devuelve 202 pending, luego worker persiste success y findings.
Historial	Cubierto por MockMvc y Playwright: GET /api/audits lista resumen con filtros, detalle y metricas.
Seguridad	Cubierto por prueba 401 sin JWT en /api/audits y validacion de token en filtro JWT.
Servicio IA	Cubierto por pytest: modo mock, modo ai sin API key, fallback y codigo vacio.

Pruebas de Integración

Backend: Sprint1EndToEndTest cubre registro, login, POST /api/audits con JWT, persistencia asincronica de hallazgos, filtros de historial, metricas, detalle y rechazo 401 sin token. Python: tests de /analyze validan modo mock, error controlado sin API key, fallback a mock y codigo vacio con 400. Frontend: Playwright simula login, envio de codigo, hallazgos, historial y metricas.

Pruebas de IA

El servicio IA se prueba en modo mock por defecto para estabilidad y en modo ai sin credenciales para verificar error 500 controlado. Tambien se prueba AI_FALLBACK_TO MOCK=true para mantener el contrato cuando no existe API key.

14

14. Desafíos Técnicos

Decisiones y resolución

[Documentación técnica](#)

Durante el desarrollo se identificaron diversos desafíos técnicos.

Comunicación Java ↔ Python

El desafío principal fue desacoplar el ciclo asincronico: Java responde 202 de inmediato y un worker invoca Python en segundo plano. Se agregaron reintentos para errores 5xx, circuit breaker temporal y manejo de fallas que marca la auditoria como failed sin romper el contrato del frontend.

Integración con IA

La integración exigió forzar una respuesta JSON estricta. PromptBuilder inyecta el JSON Schema de AnalyzeResponse y AIAnalysisService usa parse con response_format para validar la salida; si el proveedor rechaza, no responde o devuelve un formato invalido, el servicio genera una respuesta failed controlada.

Persistencia

La persistencia se resolvió con JPA/Hibernate y relaciones bidireccionales controladas. Audit guarda el código y metadatos, Finding se asocia con cascade/orphanRemoval, y los repositorios usan consultas agregadas para resumen, filtros y métricas por usuario.

15

15. Lecciones Aprendidas

Aprendizajes del desarrollo

Documentación técnica



La separación de responsabilidades facilita probar y evolucionar la solución: React se concentra en experiencia, Java en seguridad/persistencia/orquestación y Python en IA. También se aprendió la importancia de contratos estables, pruebas con mocks, estados asíncronos, límites de costo y manejo explícito de errores externos.

16

16. Conclusiones

Resultado final

Documentación técnica



El desarrollo de la Plataforma de Auditoría de Código basada en Inteligencia Artificial permitió aplicar de forma práctica los conceptos abordados durante la materia Programación de Vanguardia, integrando tecnologías modernas, arquitecturas distribuidas y servicios de Inteligencia Artificial en una solución funcional y escalable.

La plataforma propuesta resuelve la necesidad de asistir a desarrolladores y estudiantes en la detección temprana de vulnerabilidades, errores de programación y oportunidades de mejora, proporcionando además explicaciones pedagógicas y sugerencias de refactorización que favorecen el aprendizaje continuo.

Desde el punto de vista técnico, la solución se construyó sobre una arquitectura de microservicios compuesta por un frontend React, un backend Java Spring Boot, un servicio de análisis desarrollado en Python FastAPI y una base de datos SQL Server.

La incorporación de Inteligencia Artificial mediante modelos de lenguaje demostró ser una alternativa viable para enriquecer los procesos tradicionales de revisión de código.

Como líneas de evolución futura, se identifican oportunidades de mejora tales como la integración con repositorios Git, el análisis de proyectos completos, la incorporación de análisis estático especializado y la integración con pipelines CI/CD.

En conclusión, el trabajo desarrollado demuestra la viabilidad de combinar arquitecturas modernas e Inteligencia Artificial para construir herramientas de asistencia al desarrollo de software.

17

17. Anexos

Material complementario

Documentación técnica



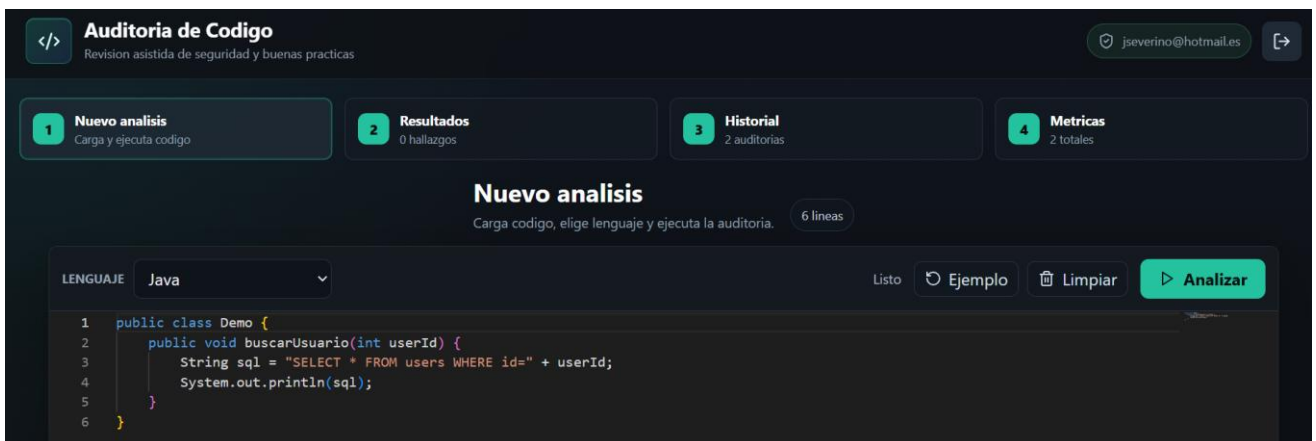
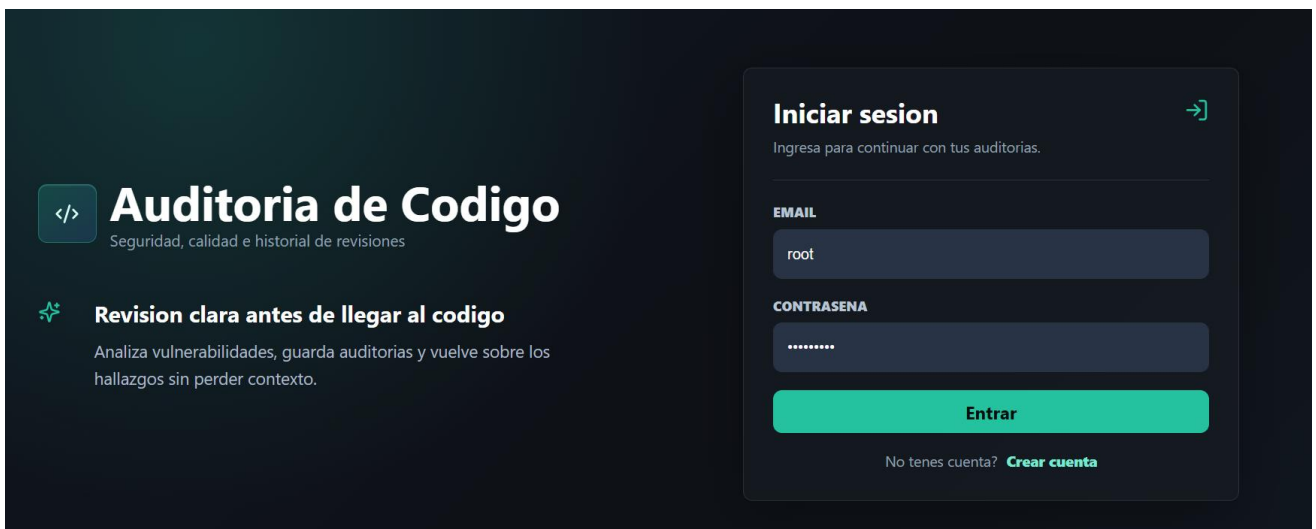
Repositorio

Url: <https://github.com/lucassaputo/programacionVanguardia>

Rama: main (Producción)

Url Aplicación: <https://intimate-actively-escargot.ngrok-free.app/audit/>

Capturas de Pantalla



</>

Auditoria deCodigo

Revisión asistida de seguridad y buenas practicas

jseverino@hotmail.es

➔

1 Nuevo analisis
Carga y ejecuta código

2 Resultados
1 hallazgo

3 Historial
3 auditorias

4 Metricas
3 totales

Resultado de auditoria

Revisa riesgo, hallazgos y recomendaciones accionables.

Analisis completado

Hallazgos

1 hallazgo

RIESGO GENERAL
critical

HALLAZGOS
1

ESTADO
Analisis completado

LENGUAJE
java

Possible SQL Injection

CRITICAL

Linea 1 - security

The code concatenates user input directly into SQL.

RECOMENDACION

Use parameterized queries or PreparedStatement.

Ver en editor

Explicacion pedagogica

SQL Injection ocurre cuando datos externos se concatenan directamente en una consulta SQL, permitiendo que un atacante modifique la intencion original de la consulta.

Codigo sugerido

PreparedStatement stmt = connection.prepareStatement("SELECT * FROM users WHERE id = ?");
stmt.setInt(1, userId);

Volver al editor

</>

Auditoria deCodigo

Revisión asistida de seguridad y buenas practicas

jseverino@hotmail.es

➔

1 Nuevo analisis
Carga y ejecuta código

2 Resultados
1 hallazgo

3 Historial
3 auditorias

4 Metricas
3 totales

Auditorias guardadas

Consulta auditorias anteriores y recupera sus resultados.

3 visibles

Historial

3

Q Buscar por lenguaje, riesgo o estado

Todos

Todos

JAVA

2/6/2026, 06:01:14

Analisis completado

critical

1 hallazgo

➔

JAVA

31/5/2026, 06:12:05

Analisis completado

critical

1 hallazgo

➔

JSON

19/5/2026, 05:12:47

Analisis completado

critical

1 hallazgo

➔

Anterior

Página 1

Siguiente

Universidad de la Ciudad de Buenos Aires

Trabajo Práctico Final

Página 29 de 30

